DSIP — Module 3

Image Transform: Frequency Domain Representation & Enhancement

Complete revision guide — every formula, numerical, analogy, and exam-line in one document.

Block	Topic	Pages
1	Foundation — what is a transform, why frequency domain	~2
2	DFT — formula, twiddle, matrix, properties, 2D DFT	~5
3	FFT — Radix-2 DIT, butterfly, bit-reversal, worked Q	~3
4	DCT, Hadamard, Walsh	~3
5	Haar and Wavelet intro (LL/LH/HL/HH sub-bands)	~2
6	PCA / Hotelling / KL transform — full 6-step recipe	~4
7	Frequency Domain Filters — Ideal, Butterworth, Gaussian, Homomorphic	~3
8	Master Cheat Sheet + Exam-line bank	~2

Block 1 — Foundation

1.1 What is a transform?

A transform is just a different way of looking at the same information. A signal can be described in the time domain (loudness at each moment) or in the frequency domain (how much bass / mid / treble it has). Both descriptions hold the same information — no information is lost. Transforms simply project a signal onto a set of basis functions; different basis functions = different transforms.

Different transforms = different lenses on the same image.

Transform	Basis function
Fourier (DFT)	Sines and cosines
DCT	Cosines only
Walsh / Hadamard	Square waves (+1, −1)
Haar	Step-like wavelets
PCA / KL	Eigenvectors of the data itself

1.2 Why use the frequency domain? (2 reasons)

(i) Mathematical convenience — convolution in time domain becomes simple multiplication in frequency domain:

$$x(n) * h(n) \leftrightarrow_{DFT} X(k) \cdot H(k)$$

(ii) Information extraction — some features are invisible in time domain but obvious in frequency. A noisy image looks messy spatially, but in the frequency domain the noise sits at high frequencies — easy to filter out.

Mnemonic / Memory hook:** **M**ath is easier, **M**ore info visible → **MM

1.3 Four categories of image transforms

#	Category	Basis type	Examples
1	Sinusoidal orthogonal	sine / cosine	DFT, DCT
2	Non-sinusoidal orthogonal	square waves, wavelets	Walsh, Hadamard, Haar, Wavelet
3	Data-dependent	eigenvectors of data	KL transform (PCA / Hotelling), SVD
4	Directional	directional features	Hough, Radon, Ridgelet, Contourlet

***Mnemonic / Memory hook:** **S**ines, **S**quares, **S**tatistics, **S**hapes → 4 S's*

Exam line: KL transform (PCA) has the best energy compaction among all linear transforms.

1.4 Key principle to lock in

Transforms do NOT change the information content of a signal. They only change its representation. Taking the DFT of an image does not lose information — you can always invert back

to the original.

Block 2 — Discrete Fourier Transform (DFT)

2.1 What DFT does

Given a discrete signal of N samples x(n), DFT computes how much of each frequency component is present. Output X(k) is N complex numbers; magnitude |X(k)| tells you how much of that frequency exists, phase ∠X(k) tells you where that wave starts. DFT takes N time samples → N frequency coefficients (same info, different view).

2.2 The DFT & IDFT formulas

DFT:

$$X(k) = \sum_{n=0 \text{ to } N-1} x(n) \cdot e^{-j2\pi kn/N}, \quad k = 0, 1, \ldots, N-1$$

IDFT:

$$x(n) = (1/N) \sum_{k=0 \text{ to } N-1} X(k) \cdot e^{+j2\pi kn/N}$$

Three things change between DFT and IDFT: (1) sign in exponent flips –j → +j, (2) factor 1/N appears in front of IDFT, (3) sum runs over k instead of n.

2.3 Twiddle factor

$$W_N = e^{-j2\pi/N}$$

So DFT becomes:

$$X(k) = \sum x(n) \cdot W_N^{kn}$$

Twiddle factors are **periodic**. For N = 4:

Power	Value	Why
W_4^0	1	$e^0 = 1$
W_4^1	–j	$e^{-j\pi/2} = 0 - j$
W_4^2	–1	$e^{-j\pi} = -1$
W_4^3	+j	$e^{-j3\pi/2} = +j$
W_4^4	1 (= W_4^0)	cycle repeats

Mnemonic / Memory hook: *For N = 4 memorize the cycle:* ***1, –j, –1, +j*** *— it appears in every 4-point DFT/FFT problem.*

2.4 The DFT matrix (4×4)

Entry at row k, column n is W^{kn}. After substituting and reducing mod 4:

$$W4 = \begin{vmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{vmatrix}$$

Then: $\mathbf{X = W_4 \cdot x}$ (multiply matrix by input column vector).

Mnemonic / Memory hook: *Row 0: all 1's. Row 1: the twiddle cycle* ***1, –j, –1, j****. Row 2: alternating* ***1, –1, 1, –1****. Row 3: reverse of row 1. The matrix is* ***symmetric along the diagonal****.*

2.5 Worked numerical — DFT of x(n) = {0, 1, 2, 3}

X(0) = (1)(0) + (1)(1) + (1)(2) + (1)(3) = **6**
X(1) = (1)(0) + (–j)(1) + (–1)(2) + (j)(3) = 0 – j – 2 + 3j = **–2 + 2j**
X(2) = (1)(0) + (–1)(1) + (1)(2) + (–1)(3) = **–2**
X(3) = (1)(0) + (j)(1) + (–1)(2) + (–j)(3) = **–2** – **2j**

Result: X(k) = {6, –2+2j, –2, –2–2j}

Sanity check: X(0) = sum of all input samples (the 'DC component'). If your X(0) doesn't equal Σx(n), you've made an arithmetic error. **Always verify this in exams.**

Conjugate symmetry: notice X(1) and X(3) are complex conjugates. For real input, **X(N–k) = X*(k)**. *Exam hack:* compute X(0), X(1), X(2), then write X(3) by taking the conjugate of X(1).

2.6 Worked numerical — DFT of x(n) = {1, 2, 3, 4}

X(0) = 1 + 2 + 3 + 4 = **10** (sanity check passes)
X(1) = 1 – 2j – 3 + 4j = **–2 + 2j**
X(2) = 1 – 2 + 3 – 4 = **–2**
X(3) = conjugate of X(1) = **–2 – 2j**

Result: X(k) = {10, –2+2j, –2, –2–2j}

Pattern observation: compared to the previous example (added +1 to every sample), only X(0) changed. Adding a constant only affects the DC component — this is a preview of the linearity property.

2.7 Properties of DFT (the 8 — 3-2-3 grouping)

Group 1: The "obvious" ones (3)

1. Linearity: $a \cdot x_1(n) + b \cdot x_2(n) \leftrightarrow a \cdot X_1(k) + b \cdot X_2(k)$

2. Periodicity: X(k + N) = X(k). Output repeats every N samples.

3. Conjugate symmetry: for real x(n), X(N–k) = X*(k). Second half mirrors the first.

Group 2: The shift pair (2)

4. Circular time shift: $x((n - n_0))_N \leftrightarrow X(k) \cdot W_N^{k \cdot n_0}$. Magnitude unchanged, only phase shifts.

5. Circular frequency shift: $x(n) \cdot W_N^{-k_0 n} \leftrightarrow X((k - k_0))_N$. Dual of property 4.

Mnemonic / Memory hook: *Time shift → phase twist. Frequency shift → time twist. Symmetric pair.*

Group 3: The operations (3)

6. Circular convolution: $x_1(n) \otimes x_2(n) \leftrightarrow X_1(k) \cdot X_2(k)$. *The most important property — foundation of frequency domain filtering.*

7. Multiplication: $x_1(n) \cdot x_2(n) \leftrightarrow (1/N)\,[X_1(k) \otimes X_2(k)]$. Mirror of property 6.

8. Parseval's theorem (energy conservation):

$$\Sigma_n |x(n)|^2 = (1/N)\ \Sigma_k |X(k)|^2$$

Energy is preserved — whether measured in time or frequency, you get the same value.

Mnemonic / Memory hook: 3-2-3 grouping — LPS / TF / CMP:
*3 obvious: **L**inear, **P**eriodic, **S**ymmetric*
*2 shifts: **T**ime-shift, **F**requency-shift*
*3 operations: **C**onvolution, **M**ultiplication, **P**arseval*

2.8 2D DFT (for images)

$$F(u, v) = \Sigma_x\ \Sigma_y\ f(x, y) \cdot e^{-j2\pi(ux/M + vy/N)}$$

Separability property: 2D DFT = 1D DFT applied to rows, then 1D DFT applied to columns (order doesn't matter). This is what makes 2D DFT computationally feasible.

Procedure: apply 1D DFT to each row of f → intermediate F′. Apply 1D DFT to each column of F′ → final F(u, v).

Matrix form:

$$F = W_M \cdot f \cdot W_N$$

For a square image (M = N): $\mathbf{F = W_N \cdot f \cdot W_N}$ — the “sandwich” pattern.

Mnemonic / Memory hook: Sandwich the image between two DFT matrices: F = W · f · W

Symmetric vs asymmetric transformation matrices (general rule that applies to all transforms):
If T is symmetric: $\mathbf{F = T \cdot f \cdot T}$. If T is asymmetric: $\mathbf{F = T \cdot f \cdot T^T}$ (use transpose).

Block 3 — Fast Fourier Transform (FFT, Radix-2 DIT)

3.1 Why FFT exists

FFT is not a new transform — it computes the same DFT, but much faster, by avoiding redundant calculations. Direct DFT requires N^2 complex multiplications. FFT reduces this to $(N/2)\log_2 N$. The speedup grows dramatically with N.

N	DFT mults (N^2)	FFT mults ($(N/2)\log_2 N$)	Speedup
8	64	12	~5x
64	4,096	192	~21x
1024	1,048,576	5,120	~205x

Exam line: FFT reduces computational complexity from $O(N^2)$ to $O(N \log_2 N)$.

3.2 The two key tricks (twiddle factor properties)

FFT exploits two properties of W_N:

(1) **Symmetry:** $W_N^{k+N/2} = -W_N^k$

(2) **Periodicity:** $W_N^{k+N} = W_N^k$

3.3 Divide and conquer

Split the N-point input into **even-indexed samples** {x(0), x(2), x(4), …} and **odd-indexed samples** {x(1), x(3), x(5), …}. Each is an (N/2)-point sequence. Compute the smaller DFTs, combine with twiddle factors, get the full N-point DFT.

Splitting on time indices = **Decimation in Time (DIT)**. Splitting on frequency indices = Decimation in Frequency (DIF). Your syllabus covers DIT only.

Recursive structure for N = 8:

Size 8 DFT → two Size 4 DFTs → four Size 2 DFTs.

Exam line: For Radix-2 FFT, N must be a power of 2 ($N = 2^m$). Even-but-not-power-of-2 (e.g., N = 6, 10) does not work.

Number of stages = $\log_2 N$. So N = 4 has 2 stages, N = 8 has 3 stages.

3.4 The butterfly operation

The single building block of FFT. Takes 2 inputs, produces 2 outputs, uses 1 multiplication:

```
Top output A = a + W · b
Bottom output B = a − W · b
```

Compute W·b once, then add for top, subtract for bottom. One multiplication, two outputs — this is where the savings come from. The crossing X-pattern in the diagram looks like wings, hence "butterfly."

3.5 Bit reversal (input order for DIT-FFT)

In DIT-FFT, input goes in **bit-reversed order**; output comes out in natural order.

For N = 4:

Natural	Binary	Reversed	New index
0	00	00	0
1	01	10	2
2	10	01	1
3	11	11	3

Bit-reversed order for N = 4: 0, 2, 1, 3 (just “swap the middle two” — 0 and 3 are palindromes)

For N = 8:

Natural	0	1	2	3	4	5	6	7
Binary	000	001	010	011	100	101	110	111
Reversed	000	100	010	110	001	101	011	111
New index	0	4	2	6	1	5	3	7

Bit-reversed order for N = 8: 0, 4, 2, 6, 1, 5, 3, 7

3.6 Worked numerical — 4-point DIT-FFT of x(n) = {0, 1, 2, 3}

Step 1: Bit-reverse the input → x(0)=0, x(2)=2, x(1)=1, x(3)=3.

Step 2: Stage 1 — two butterflies, each with twiddle $W_4^0 = 1$.

Butterfly 1 (inputs 0, 2): A = 0 + 1·2 = **2**, B = 0 – 1·2 = **–2**
Butterfly 2 (inputs 1, 3): C = 1 + 1·3 = **4**, D = 1 – 1·3 = **–2**
After Stage 1: {A, B, C, D} = {2, –2, 4, –2}

Step 3: Stage 2 — cross-pair (A,C) using W_4^0=1, and (B,D) using W_4^1=–j.

Top butterfly (W=1): X(0) = 2 + 1·4 = **6**, X(2) = 2 – 1·4 = **–2**
Bottom butterfly (W=–j): X(1) = –2 + (–j)(–2) = **–2 + 2j**, X(3) = –2 – (–j)(–2) = **–2 – 2j**

Result: X(k) = {6, –2+2j, –2, –2–2j} (matches the direct DFT result)

3.7 Worked numerical — 4-point DIT-FFT of x(n) = {1, 2, 3, 4}

Bit-reversed: x(0)=1, x(2)=3, x(1)=2, x(3)=4.

Stage 1: A = 1+3 = **4**, B = 1–3 = **–2**, C = 2+4 = **6**, D = 2–4 = **–2**
Stage 2:
X(0) = A + 1·C = 4 + 6 = **10**
X(1) = B + (–j)·D = –2 + 2j = **–2 + 2j**
X(2) = A – 1·C = 4 – 6 = **–2**
X(3) = B – (–j)·D = –2 – 2j = **–2 – 2j**

Result: X(k) = {10, –2+2j, –2, –2–2j}

3.8 Why those specific twiddles in each stage? (deep intuition)

Stage 1 does 2-point DFTs. The 2-point DFT of {a, b} gives X(0) = a + b, X(1) = a – b. The "–1" is hidden inside the butterfly's subtraction — that's why Stage 1 looks like $W_4^0 = 1$.

Stage 2 combines into a 4-point DFT using X(k) = G(k) + W_4^k·H(k). k = 0, 1, 2, 3 needs four twiddles, but the symmetry trick $W^{k+N/2} = -W^k$ means k = 2 is just –(k = 0), and k = 3 is just –(k = 1). So we only need **two physical multiplications** ($W_4^0 = 1$ and $W_4^1 = -j$) — the "minus" comes free from the butterfly's bottom output.

3.9 Common exam questions on FFT

Question type	What to do
Find FFT of N-point sequence	Bit-reverse, do $\log_2 N$ stages of butterflies
Draw signal flow graph	Sketch butterflies + label twiddles
Compare DFT vs FFT operations	DFT: N^2. FFT: $(N/2)\log_2 N$
Why must N be power of 2?	Recursive halving demands it

Why bit reversal?	Even-odd splitting cascades into bit-reversed input order

Block 4 — DCT, Hadamard, Walsh

4.1 Discrete Cosine Transform (DCT)

DFT uses sines AND cosines. DCT uses **only cosines** — basis functions are real-valued. Cosines match natural images very well, leading to **excellent energy compaction**: most energy concentrates in a few low-frequency coefficients. This is why **JPEG uses DCT**.

1D DCT formula:

$$C(k) = \alpha(k) \sum_{n=0 \text{ to } N-1} x(n) \cdot \cos[(2n+1)k\pi / 2N]$$

Normalization factor:

$$\alpha(k) = \sqrt{(1/N)} \quad \text{if } k = 0$$
$$\alpha(k) = \sqrt{(2/N)} \quad \text{if } k = 1, 2, \ldots, N-1$$

1D IDCT formula:

$$x(n) = \sum_{k} \alpha(k) \cdot C(k) \cdot \cos[(2n+1)k\pi / 2N]$$

DCT matrix for N = 4 (provided in most exams — don't memorize entries):

```
       |  0.500   0.500   0.500   0.500 |
C4 =   |  0.653   0.271  -0.271  -0.653 |
       |  0.500  -0.500  -0.500   0.500 |
       |  0.271  -0.653   0.653  -0.271 |
```

Then: DCT output = $C_4 \cdot x$ (same procedure as DFT — multiply matrix by input column).

DCT properties (theory marks)

1. **Real-valued** — no complex numbers.
2. **Orthogonal.**
3. **Separable** (2D DCT = 1D DCT on rows, then on columns).
4. **Best energy compaction** for natural images.
5. Used in **JPEG, MPEG, H.264**.
6. DCT can be computed via FFT (a derivation result, useful trivia).

Exam line: DCT has better energy compaction than DFT for natural images, and produces real-valued output, which is why JPEG uses DCT.

4.2 Hadamard Transform

Simplest possible transform. Matrix has only **+1 and −1**. No multiplications — only additions and subtractions. Extremely fast in hardware.

Sylvester's recursive construction:

```
       |  1   1 |
H2 =   |  1  -1 |

        |  H_N    H_N  |
H_2N =  |              |
        |  H_N   -H_N  |
```

Building H_4 from H_2 — top-left, top-right, bottom-left = H_2; bottom-right = $-H_2$:

```
        |  1   1   1   1 |
H4 =    |  1  -1   1  -1 |
        |  1   1  -1  -1 |
        |  1  -1  -1   1 |
```

Hadamard transform formula:

$$Y = (1/\sqrt{N}) \cdot H_N \cdot x$$

For 2D images:

$$Y = (1/N) \cdot H_N \cdot f \cdot H_N$$

Hadamard properties

1. **Real-valued and orthogonal.**
2. **Symmetric:** $H_N = H_N^T$.
3. Size **must be a power of 2.**
4. **Self-inverse:** $H_N \cdot H_N = N \cdot I \rightarrow$ inverse Hadamard = Hadamard (with 1/N normalization).
5. **No multiplications** in computation.

Exam line: Hadamard transform is its own inverse (up to 1/N scaling) — favorite 2-mark question.

4.3 Worked numerical — 4-point Hadamard of x = {1, 2, 3, 4}

Compute $H_4 \cdot x$ (no normalization):

Y(0) = (1)(1) + (1)(2) + (1)(3) + (1)(4) = **10** (sum check)
Y(1) = (1)(1) + (–1)(2) + (1)(3) + (–1)(4) = 1 – 2 + 3 – 4 = **–2**
Y(2) = (1)(1) + (1)(2) + (–1)(3) + (–1)(4) = 1 + 2 – 3 – 4 = **–4**
Y(3) = (1)(1) + (–1)(2) + (–1)(3) + (1)(4) = 1 – 2 – 3 + 4 = **0**

Result: Y = {10, –2, –4, 0}
If 1/√N normalization required → multiply by 1/2 → {5, –1, –2, 0}.

4.4 Worked numerical — 4-point Hadamard of x = {2, 4, 6, 8}

Y(0) = 2 + 4 + 6 + 8 = **20**
Y(1) = 2 – 4 + 6 – 8 = **–4**
Y(2) = 2 + 4 – 6 – 8 = **–8**
Y(3) = 2 – 4 – 6 + 8 = **0**

Result: Y = {20, –4, –8, 0} (= 2 × previous result — linearity confirmed!)

4.5 Walsh Transform

Walsh = **Hadamard with rows reordered by sequency** (number of sign changes per row). This makes Walsh more analogous to DFT (where rows are naturally ordered by frequency).

Sequency of each H_4 row:

H4 row	Pattern	Sign changes (sequency)
Row 1	1, 1, 1, 1	0
Row 2	1, -1, 1, -1	3
Row 3	1, 1, -1, -1	1
Row 4	1, -1, -1, 1	2

Walsh matrix (sort H_4 rows by sequency 0, 1, 2, 3):

```
        |  1   1   1   1 |   <- sequency 0
W4 =    |  1   1  -1  -1 |   <- sequency 1
        |  1  -1  -1   1 |   <- sequency 2
        |  1  -1   1  -1 |   <- sequency 3
```

Walsh transform of x = {1, 2, 3, 4} → Y = {10, –4, 0, –2}.
Same elements as Hadamard {10, –2, –4, 0}, just reshuffled order — since rows are reordered, output coefficients are reordered too. Information identical.

Walsh vs Hadamard summary

Feature	Hadamard	Walsh
Entries	±1	±1

Construction	Sylvester recursion	Hadamard rows sorted by sequency
Order of rows	Natural recursion	By sequency (low $\rightarrow$ high)
Real / complex	Real	Real
Use	General	Communication signal analysis

Block 5 — Haar Transform & Wavelet Intro

5.1 Haar Transform

The first wavelet transform (Alfréd Haar, 1909). Like Hadamard it's simple, but with a special property: **it captures both location AND frequency information**. DFT/DCT tell you what frequencies exist; Haar (and wavelets in general) tell you both *what* and *where*.

Haar matrix for N = 4:

```
                 |  1      1      1      1    |
H_haar(4) =      |  1      1     -1     -1    |  * (1/2)
                 |  sqrt(2)  -sqrt(2)   0     0    |
                 |  0      0     sqrt(2)  -sqrt(2)|
```

What each row captures:

Row	Pattern	Captures
Row 1	1, 1, 1, 1	Average / overall trend (low frequency)
Row 2	1, 1, -1, -1	Coarse difference (first half vs second half)
Row 3	√2, -√2, 0, 0	Local detail (left side only — zeros mean it "sees" only that part)
Row 4	0, 0, √2, -√2	Local detail (right side only)

***Mnemonic / Memory hook: Order from top to bottom: A–D–L–R** (Average, Difference, Left-detail, Right-detail). The zeros in rows 3 and 4 are why Haar captures **location**.*

Haar transform formula: $Y = H_{haar} \cdot x$

Haar properties

1. Real-valued and orthogonal.
2. Captures **frequency AND location**.
3. **Foundation of all wavelet transforms** (Daubechies, Coiflets, etc., are generalizations).
4. Used in **JPEG 2000** image compression.
5. Enables **multi-resolution analysis** — see signal at different scales.

5.2 Wavelet Transform (intro)

The big idea: DFT tells you what frequencies exist. Wavelets tell you what frequencies exist AND where they exist.

Why this matters for images: a smooth blue sky is low-freq + large area; a sharp building edge is high-freq + localized. DFT mixes these together. Wavelet separates them by location — perfect for image compression and edge detection.

Multi-resolution analysis

Wavelets analyze signals at multiple resolutions (zoom levels):

- Low resolution → captures overall shape
- High resolution → captures fine details

This matches how human vision works (big shapes first, then details).

2D wavelet decomposition — the 4 sub-bands

Sub-band	Pass type (rows / cols)	Contains
LL	Low / Low	Approximation (smooth version of image)
LH	Low / High	Horizontal edges
HL	High / Low	Vertical edges
HH	High / High	Diagonal edges

Exam line: 2D wavelet decomposition gives 4 sub-bands: LL (approx), LH (horizontal), HL (vertical), HH (diagonal).

Wavelet vs DFT/DCT — the comparison question

Feature	DFT / DCT	Wavelet
Frequency info	Yes	Yes
Location info	No	Yes
Resolution	Single	Multi-resolution
Best for	Stationary signals	Non-stationary (real images)
Used in	JPEG (DCT)	JPEG 2000 (wavelet)

Exam line: Wavelets give time-frequency localization + multi-resolution analysis — advantages over Fourier.

Applications of wavelet transform

1. JPEG 2000 image compression. 2. Edge detection. 3. Denoising (removes noise while preserving edges). 4. Medical imaging (MRI, CT). 5. FBI fingerprint compression.

Block 6 — PCA / Hotelling / KL Transform

6.1 What PCA does

Given multi-dimensional data, PCA finds new axes (**principal components**) that align with directions of **maximum variance**. Most variance → most information. By keeping only the top-k principal components, you can reduce dimensionality with minimal information loss.

Three names, same thing:

Name	Field
PCA (Principal Component Analysis)	Statistics, ML
Hotelling Transform	Image processing
KL Transform (Karhunen-Loève)	Signal processing

Exam line: KL transform has the best energy compaction among all linear transforms — because it computes its basis directly from the data's statistics.

Trade-off: KL/PCA basis is data-dependent (must be computed for every new dataset). DCT's basis is fixed — faster, but slightly less optimal.

6.2 The 6-step PCA recipe

Step	Action	Why
1	Compute mean vector	Need to center data
2	Subtract mean from each data point	PCA needs zero-mean data
3	Compute covariance matrix C	Captures how features vary together
4	Find eigenvalues from det(C - λI) = 0	Tell us how much variance along each direction
5	Find eigenvectors; normalize each to unit length	These are the principal components
6	Stack eigenvectors as rows of A, sorted by largest λ first	The transformation matrix

***Mnemonic / Memory hook: MCCEEP** — **M**ean, **C**enter, **C**ovariance, **E**igenvalues, **E**igenvectors, **P**roject ("Many Cats Can't Eat Easy Pizza")*

6.3 Key formulas

Variance & covariance (use 1/N or 1/(N-1) — both accepted):

$$\mathrm{Var}(X) = (1/N)\ \Sigma\ (x_i - \mathrm{mean}_x)^2$$
$$\mathrm{Var}(Y) = (1/N)\ \Sigma\ (y_i - \mathrm{mean}_y)^2$$
$$\mathrm{Cov}(X, Y) = (1/N)\ \Sigma\ (x_i - \mathrm{mean}_x)(y_i - \mathrm{mean}_y)$$

Covariance matrix (2D case):

```
        |  Var(X)    Cov(X,Y) |
  C  =  |                     |
        |  Cov(X,Y)  Var(Y)   |
```

Eigenvalue equation:

$$\det(C - \lambda I) = 0$$

Eigenvector equation: for each λ, solve $(C - \lambda I) \cdot v = 0$, then normalize: $e = v / |v|$

Vector normalization (any dimension):

$$|v| = \sqrt{(v_1^2 + v_2^2 + \ldots + v_n^2)} \qquad e = v / |v|$$

Transformation:

$$y = A \cdot (x - m) \quad \text{(project to PCA coords)}$$
$$x = m + A^T \cdot y \quad \text{(reconstruct)}$$

6.4 Three crucial sanity checks

1. **Centered data sums to 0** in each dimension → catches arithmetic errors in Step 2.
2. **Sum of eigenvalues = trace of C** (sum of diagonal); **product of eigenvalues = det of C** → catches errors in Step 4.
3. **Eigenvectors are orthogonal** ($e_1 \cdot e_2 = 0$) → catches errors in Step 5.

6.5 Full worked numerical — PCA of 4 data points

Data: $x_1 = (2, 1)$, $x_2 = (3, 2)$, $x_3 = (4, 3)$, $x_4 = (5, 4)$.

Step 1 — Mean:
$\text{mean}_x = (2+3+4+5)/4 = 3.5$ $\text{mean}_y = (1+2+3+4)/4 = 2.5$ **m = (3.5, 2.5)**

Step 2 — Center:
$x_1 - m = (-1.5, -1.5)$ $x_2 - m = (-0.5, -0.5)$
$x_3 - m = (0.5, 0.5)$ $x_4 - m = (1.5, 1.5)$
Sum check: x-deviations $-1.5-0.5+0.5+1.5 = 0$ ✓

Step 3 — Covariance matrix:
$\text{Var}(X) = (1/4)(2.25 + 0.25 + 0.25 + 2.25) = 1.25$
$\text{Var}(Y) = 1.25$ (same, since x and y move together)
$\text{Cov}(X,Y) = (1/4)(2.25 + 0.25 + 0.25 + 2.25) = 1.25$

$C = [[1.25, 1.25], [1.25, 1.25]]$

Step 4 — Eigenvalues:
$(1.25 - \lambda)^2 - 1.5625 = 0 \rightarrow 1.25 - \lambda = \pm 1.25$
$\lambda_1 = 2.5$, $\lambda_2 = 0$
Sanity check: $2.5 + 0 = \text{trace} = 2.5$ ✓
($\lambda_2 = 0$ means our 2D data is actually 1D in disguise!)

Step 5 — Eigenvectors:
For $\lambda_1 = 2.5$: $(C - \lambda_1 I)v = 0$ gives $v_1 = v_2 \rightarrow v = (1,1) \rightarrow \mathbf{e_1 = (1/\sqrt{2})(1, 1) \approx (0.707, 0.707)}$
For $\lambda_2 = 0$: gives $v_1 = -v_2 \rightarrow v = (1,-1) \rightarrow \mathbf{e_2 = (1/\sqrt{2})(1, -1) \approx (0.707, -0.707)}$
Orthogonality: $e_1 \cdot e_2 = 0$ ✓

Step 6 — Transformation matrix A (eigenvectors as rows, λ_1 first):
$A = [[0.707, 0.707], [0.707, -0.707]]$

To project: $y = A \cdot (x - m)$. Each x_i projected onto PCA coordinates — the second coordinate is always 0 because $\lambda_2 = 0$ (data is 1D in disguise).

6.6 Worked numerical 2 — eigenvalues for non-trivial case

Data: $x_1=(1,2)$, $x_2=(3,3)$, $x_3=(3,5)$, $x_4=(5,4)$

Mean: $m = (3, 3.5)$
Centered: $(-2, -1.5)$, $(0, -0.5)$, $(0, 1.5)$, $(2, 0.5)$

Covariance: $\text{Var}(X)=2$, $\text{Var}(Y)=1.25$, $\text{Cov}(X,Y)=1$
$C = [[2, 1], [1, 1.25]]$

Characteristic equation: $(2-\lambda)(1.25-\lambda) - 1 = 0$
$\rightarrow \lambda^2 - 3.25\lambda + 1.5 = 0$
Discriminant $= 10.5625 - 6 = 4.5625$; $\sqrt{4.5625} \approx 2.136$
$\lambda_1 \approx \mathbf{2.693}$, $\lambda_2 \approx \mathbf{0.557}$
Sanity: $2.693 + 0.557 = 3.25 = \text{trace}$ ✓ ; $2.693 \times 0.557 \approx 1.5 = \det$ ✓

Eigenvectors (after solving and normalizing):
$e_1 \approx (0.822, 0.569)$, $e_2 \approx (0.569, -0.822)$
Orthogonality check: $0.822 \cdot 0.569 + 0.569 \cdot (-0.822) = 0$ ✓

Variance retained by keeping only e_1: 2.693 / 3.25 ≈ **82.9%**

6.7 What to do AFTER getting A (Steps 7+)

Project: $y = A \cdot (x - m)$ — data in new PCA coordinates.

Reduce dimensions: use only top-k rows of A (corresponding to k largest λ) — drop dimensions with small λ for compression.

Reconstruct: $x_{recon} = m + A^T \cdot y$ — go back to original space. Full A → perfect reconstruction. Reduced A → some error, equal to the variance discarded.

% variance retained = (kept eigenvalues) / (sum of all eigenvalues) × 100

Exam line: If $\lambda_2 \ll \lambda_1$, most variance lies along e_1 — the data can be reduced to 1D with minimal information loss. This is energy compaction.

Block 7 — Frequency Domain Filters

7.1 The universal pipeline

Every frequency-domain filter follows this 3-step pipeline:

$$f(x,y) \xrightarrow{\text{DFT}} F(u,v) \xrightarrow{\times H(u,v)} G(u,v) \xrightarrow{\text{IDFT}} g(x,y)$$

Take DFT of input image. Multiply by filter function H(u,v). Take IDFT → filtered image. **Different H = different filter.**

***Mnemonic / Memory hook: D-M-I** → DFT, Multiply, IDFT*

Why filter in frequency domain? Because convolution in spatial domain = multiplication in frequency domain. Multiplication is much faster than convolution, especially for large filters.

7.2 Distance function and cutoff

D(u,v) = distance from the center of the spectrum to point (u,v):

$$D(u, v) = \sqrt{[(u - M/2)^2 + (v - N/2)^2]}$$

$\mathbf{D_0}$ = cutoff frequency — the radius (in frequency space) at which the filter "kicks in." This is a parameter you choose.

7.3 Low-pass vs high-pass intuition

Filter	Keeps	Removes	Effect
Low-pass (LPF)	Low frequencies (smooth regions)	High frequencies (edges, noise)	Blurs / smooths / denoises
High-pass (HPF)	High frequencies (edges, detail)	Low frequencies (smooth regions)	Sharpens / enhances edges

Mnemonic / Memory hook: LPF Lulls (smooths), HPF Highlights (sharpens edges)

7.4 The four main filter formulas

Filter	Formula	Notes
Ideal LPF	$H = 1$ if $D \le D_0$ else 0	Sharpest cutoff; severe ringing
Butterworth LPF	$H = 1 / [1 + (D/D_0)^{2n}]$	Smooth; n controls sharpness; $H(D_0) = 0.5$
Ideal HPF	$H = 0$ if $D \le D_0$ else 1	Same problem as IHLP — severe ringing
Butterworth HPF	$H = 1 / [1 + (D_0/D)^{2n}]$	D and D_0 swap places vs BLPF

Pattern:

- **LPF**: D in numerator → bigger D = closer to 0 = block high frequencies
- **HPF**: D in denominator → bigger D = closer to 1 = pass high frequencies

7.5 The HPF = 1 - LPF trick

$$H_{HP}(u, v) = 1 - H_{LP}(u, v)$$

Why it works:

Frequency	LPF says	1 - LPF gives	HPF should say
Low ($D < D_0$)	1 (pass)	0	0 (block) ✓
High ($D > D_0$)	0 (block)	1	1 (pass) ✓

HPF wants the opposite of LPF; subtracting from 1 flips every keep/block decision.

7.6 Effect of order n in Butterworth

Small n (e.g., n=1) → very smooth rolloff, very little ringing.
Large n (e.g., n=10) → sharper rolloff, approaches ideal filter, ringing starts.
n → ∞ → becomes ideal filter.
At D = D_0: H = 1/(1+1) = **0.5** (always, regardless of n).

7.7 Gaussian filters (also good to know)

Bonus filter your PDF mentions. Gaussians have **no ringing at all** — smoothest filter possible.

Gaussian LPF (GLPF):

$$H(u, v) = \exp[-D^2(u,v) / (2 D_0^2)]$$

Gaussian HPF (GHPF):

$$H(u, v) = 1 - \exp[-D^2(u,v) / (2 D_0^2)]$$

Gaussians don't ring because their inverse Fourier transform is also a Gaussian (no oscillating sinc).

7.8 Band-pass and band-reject filters (1-line each)

Band-pass: keeps a moderate frequency range; rejects very low and very high. Useful for **edge enhancement while reducing noise**.
Band-reject (notch): opposite — rejects a specific band, e.g. removing periodic noise.

7.9 Homomorphic filtering

The problem it solves: images can be modeled as **illumination × reflectance**:

$$f(x, y) = i(x, y) \cdot r(x, y)$$

where i(x,y) = illumination (slow-varying, low frequency) and r(x,y) = reflectance (fast-varying, high frequency). Common case: uneven lighting. Linear filters can't easily separate two multiplied components.

The trick: take the logarithm to convert multiplication into addition:

$$\ln[f(x, y)] = \ln[i(x, y)] + \ln[r(x, y)]$$

Now they're added, not multiplied — linear filtering works on each separately.

The 5-step homomorphic pipeline

$$f(x,y) \xrightarrow{\ln} \ln[f] \xrightarrow{DFT} F(u,v) \xrightarrow{\times H(u,v)} G(u,v) \xrightarrow{IDFT} \ln[g] \xrightarrow{\exp} g(x,y)$$

Steps: (1) Take log of input. (2) Take DFT. (3) Multiply by H(u,v). (4) Take IDFT. (5) Take exp to undo the log.

The homomorphic filter shape

H(u,v) is designed to:

- **Attenuate low frequencies** → multiply by $\gamma_L < 1$ (e.g. 0.5) — reduces uneven illumination
- **Boost high frequencies** → multiply by $\gamma_H > 1$ (e.g. 2.0) — sharpens reflectance / detail

Common form (high-frequency emphasis): $H(u,v) = (\gamma_H - \gamma_L) \cdot [\, 1 - \exp(-c \cdot D^2(u,v) / D_0^2) \,] + \gamma_L$

Effects

1. **Compresses dynamic range** — fixes bright/dark unevenness.
2. **Enhances contrast** — object details pop.
3. Particularly good for images with harsh lighting (e.g., a face partially in shadow).

Exam line: Homomorphic filtering uses the logarithm to separate illumination (low freq) and reflectance (high freq), allowing low frequencies to be attenuated while high frequencies are boosted — achieving dynamic range compression and contrast enhancement simultaneously.

7.10 Summary table of all filters

Filter	Formula / Idea	Effect	Key feature
Ideal LPF	1 if $D \le D_0$, else 0	Blur	Sharpest cutoff; severe ringing
Butterworth LPF	$1 / [1 + (D/D_0)^{2n}]$	Blur	Smooth; n controls sharpness
Gaussian LPF	$\exp[-D^2/(2D_0^2)]$	Blur	No ringing
Ideal HPF	0 if $D \le D_0$, else 1	Sharpen	Severe ringing
Butterworth HPF	$1 / [1 + (D_0/D)^{2n}]$	Sharpen	Smooth edge enhancement
Gaussian HPF	$1 - \exp[-D^2/(2D_0^2)]$	Sharpen	No ringing
Band-pass	Keep mid frequencies	Edge + noise reduce	Combines LPF and HPF
Homomorphic	γ_L low, γ_H high (in log domain)	Fix lighting + sharpen	Operates in log domain

Block 8 — Master Cheat Sheet

8.1 1-page revision

Re-read this 10 minutes before the exam to refresh everything.

Foundation

• Why frequency domain: (1) math convenience (convolution → multiplication), (2) info extraction.
• 4 transform categories — **Sines, Squares, Statistics, Shapes** (4 S's).
• **KL transform = best energy compaction** among all linear transforms.

DFT

• $X(k) = \Sigma\, x(n)\, e^{-j2\pi kn/N}$; IDFT has +j and 1/N.
• Twiddle: $W_N = e^{-j2\pi/N}$; for N=4 cycle: **1, –j, –1, j**.
• **Sanity: X(0) = sum of inputs.**
• Real input → conjugate symmetric: X(N–k) = X*(k).
• 8 properties (3-2-3): Linear, Periodic, Symmetric / Time-shift, Freq-shift / Convolution, Multiplication, Parseval.
• 2D DFT: **F = W · f · W** (sandwich).

FFT (DIT Radix-2)

• **$O(N^2)$ → $O(N \log_2 N)$**.
• N must be power of 2; stages = $\log_2 N$.
• Bit reversal for N=4: **0, 2, 1, 3** (swap middle two).
• Butterfly: top = a + W·b; bottom = a – W·b.
• N=4 twiddles: Stage 1 all = 1; Stage 2 = 1 (top butterfly) and –j (bottom).

DCT

• Cosines only, real-valued, separable.
• Best energy compaction for natural images → **JPEG uses DCT**.
• $\alpha(k) = \sqrt{(1/N)}$ for k=0; $\sqrt{(2/N)}$ otherwise.

Hadamard / Walsh

• Hadamard: ±1 only, Sylvester: $H_{2N} = [[H_N, H_N], [H_N, -H_N]]$.
• $H_2 = [[1,1],[1,-1]]$; H_4 memorize.
• Self-inverse, no multiplications, real-valued.
• Walsh = Hadamard with rows sorted by sequency (number of sign changes).

Haar & Wavelet

• Haar = first wavelet; captures **frequency + location**.
• Foundation of multi-resolution analysis.
• 2D wavelet sub-bands: **LL (approx), LH (horizontal), HL (vertical), HH (diagonal)**.
• Used in JPEG 2000 (vs JPEG which uses DCT).

PCA / Hotelling / KL

• 6-step recipe: **Mean** → **Center** → **Covariance** → **Eigenvalues** → **Eigenvectors (normalize)** → **Matrix A**.

• Sanity: $\Sigma\lambda$ = trace, $\Pi\lambda$ = det, $e_1 \cdot e_2 = 0$.
• A: eigenvectors as **rows**, sorted by **largest** λ **first**.
• Transform: $y = A \cdot (x - m)$; reconstruct: $x = m + A^T \cdot y$.
• % variance retained = (kept λ) / (total λ) × 100.

Frequency domain filters

• Pipeline: **DFT** → **×H** → **IDFT**.
• D = distance from center; D_0 = cutoff.
• **ILPF**: 1 if $D \le D_0$ else 0 (sharpest, severe ringing).
• **BLPF**: $1 / [1 + (D/D_0)^{2n}]$ (smooth; n controls sharpness; $H(D_0) = 0.5$).
• **HPF = 1** – **LPF** (the flip trick).
• **BHPF**: $1 / [1 + (D_0/D)^{2n}]$ (D and D_0 swap).
• **GLPF / GHPF**: Gaussians, no ringing.
• **Homomorphic**: ln → DFT → H → IDFT → exp; separates illumination (low & $\gamma_L < 1$) from reflectance (high & $\gamma_H > 1$).

8.2 Exam-line bank (memorize these word-for-word)

1. Transforms project a signal onto a set of basis functions; different basis functions give different transforms.

2. Information content of a signal is preserved under any orthogonal transform — only the representation changes.

3. KL transform (PCA / Hotelling) has the best energy compaction among all linear transforms.

4. FFT reduces computational complexity from $O(N^2)$ to $O(N \log_2 N)$ by exploiting twiddle factor symmetry and periodicity.

5. For Radix-2 FFT, N must be a power of 2 ($N = 2^m$).

6. In DIT-FFT, input is in bit-reversed order; output is in natural order.

7. Convolution in time domain corresponds to multiplication in frequency domain (DFT property 6).

8. Parseval's theorem: energy is conserved under DFT.

9. DCT has better energy compaction than DFT for natural images, and produces real-valued output, which is why JPEG uses DCT.

10. Hadamard transform is its own inverse (up to 1/N scaling).

11. Walsh transform = Hadamard rearranged by sequency (number of sign changes per row).

12. Haar is the first wavelet transform; it captures both frequency and location information.

13. 2D wavelet decomposition produces four sub-bands: LL (approximation), LH (horizontal), HL (vertical), HH (diagonal).

14. Wavelets give time-frequency localization and multi-resolution analysis — advantages over Fourier.

15. JPEG uses DCT; JPEG 2000 uses wavelet transform.

16. PCA finds new orthogonal axes (principal components) along directions of maximum variance.

17. Eigenvectors of a covariance matrix are orthogonal (since covariance matrices are symmetric).

18. Sum of eigenvalues equals the trace of the covariance matrix; product equals its determinant.

19. Frequency domain filtering follows: DFT $\rightarrow$ multiply by $H(u,v)$ $\rightarrow$ IDFT.

20. Ideal filter has the sharpest cutoff but produces severe ringing artifacts.

21. Butterworth filter provides a smooth tradeoff between cutoff sharpness and ringing; n controls this tradeoff.

22. At cutoff frequency $D = D_0$, the Butterworth filter has gain 0.5.

23. HPF is obtained from LPF by $H_{HP} = 1 - H_{LP}$.

24. Gaussian filters produce no ringing because the inverse Fourier transform of a Gaussian is also a Gaussian.

25. Homomorphic filtering uses logarithm to separate illumination (low frequency) and reflectance (high frequency), then attenuates the former and boosts the latter — simultaneously compressing dynamic range and enhancing contrast.

8.3 Pre-exam checklist (10 minutes before the paper)

- ■ I can write the DFT formula and IDFT formula from memory.
- ■ I can compute the 4×4 W matrix without looking.
- ■ I know the 4-point twiddle cycle: 1, –j, –1, +j.
- ■ I can list the 8 DFT properties (3-2-3 grouping).
- ■ I know FFT complexity ($O(N^2)$ vs $O(N \log N)$) and why N must be a power of 2.
- ■ I know the bit-reversal pattern for N=4 (0, 2, 1, 3).
- ■ I can write the butterfly equations (top = a + W·b, bottom = a – W·b).
- ■ I know what DCT is used for (JPEG) and that it's real-valued.
- ■ I can construct H_4 from H_2 (Sylvester).
- ■ I know Walsh = Hadamard sorted by sequency.
- ■ I can name the 4 wavelet sub-bands (LL, LH, HL, HH).
- ■ I know the 6-step PCA recipe (MCCEEP).
- ■ I know the two PCA sanity checks (trace+det for λ, orthogonality for e).
- ■ I can write the 4 filter formulas (ILPF, BLPF, IHPF, BHPF).
- ■ I know HPF = 1 – LPF.
- ■ I can write the 5-step homomorphic pipeline (ln → DFT → ×H → IDFT → exp).

Made for one-day-before-exam revision. Re-read in order; trust the spaced-out worked numericals; keep the cheat sheet open during practice.

You've got this.